

Manual do Framework Agent Skills

Fases de desenvolvimento, cadeia de skills e quando usar cada uma

Agosto de 2026

Sumário

1. Introdução	1
Onde este manual se encaixa nos outros documentos do projeto	2
2. As fases (visão geral)	2
3. Tabela completa: fase, skill, quando usar	2
4. Árvore de decisão rápida (“por onde eu começo?”)	3
5. Cadeia completa — projeto do zero (greenfield)	4
Exemplo real desta sessão (greenfield dentro de um repo já existente)	4
6. Cadeia para projeto já iniciado (brownfield — o seu caso mais comum)	4
Sobreposições conhecidas (leia antes de escolher)	5
7. Comportamentos centrais (valem em toda skill, sempre)	6
8. Referência rápida (todas as skills, um-liner)	6
9. Notas específicas deste projeto (proj_fabrica-comunicacao)	7

1. Introdução

Este manual explica **como usar na prática** o framework “Agent Skills” (instalado neste repo via plugin, recarregado com /reload-plugins) e toda a família de skills “Driven Development” (Spec-Driven, Test-Driven, Doubt-Driven, Source-Driven) que vêm junto. É dirigido a quem *opera* o Claude Code neste projeto — não substitui AGENTS.md/SPEC_COMANDOS.md (que regem a Fábrica de Materiais de Comunicação em si), é um manual à parte sobre o framework de engenharia genérico que ficou disponível para qualquer trabalho de código neste repo (painel, scripts, etc.).

Cada skill encapsula um processo que engenheiros experientes seguem numa fase específica do ciclo de vida de uma mudança de código. A ideia central: **antes de começar qualquer tarefa, identifique em que fase ela está e aplique a skill correspondente** — em vez de improvisar.

Onde este manual se encaixa nos outros documentos do projeto

Tipo de documento	Pasta	Quando é criado
Manual de uso (este arquivo, MANUAL_FABRICA.md, MANUAL_EXECUTIVO.md)	manuais/	Referência de como operar algo, estável no tempo
Plano/intenção confirmado (saída de interview-me ou planejamento)	melhorias/	Toda vez que uma decisão de escopo é fechada com o operador — nunca em docs/intent/
Relatório/plano histórico de uma entrega já feita	docs/	Registro do que foi entregue, numerado sequencialmente

2. As fases (visão geral)

Definir → Planejar → Construir → Verificar → Revisar → Entregar

Cada fase tem uma ou mais skills. Nem toda tarefa passa por todas as fases — o tamanho da cadeia deve ser proporcional ao tamanho/risco da tarefa (seção 5).

3. Tabela completa: fase, skill, quando usar

Definir

- interview-me — Você (ou o pedido do operador) ainda não sabe o que quer de verdade — ask não tem quem/por quê/sucesso/restrrição claros.
- idea-refine — Já tem uma ideia (mesmo vaga), precisa gerar e comparar variações antes de decidir.
- spec-driven-development — Precisa formalizar requisitos + critério de aceite por escrito antes de codar.

Planejar

- planning-and-task-breakdown — Já tem spec/decisão, precisa quebrar em tarefas pequenas e verificáveis.

Construir

- incremental-implementation — Implementar em fatias verticais finas, testando cada uma antes de expandir.
- frontend-ui-engineering — A fatia é de UI — acessibilidade, qualidade de produção.
- api-and-interface-design — A fatia define um contrato/endpoint/limite de módulo estável.
- context-engineering — Precisa carregar o contexto certo na hora certa (evitar poluir a janela).
- source-driven-development — Precisa verificar contra documentação oficial antes de implementar (evita alucinar API).
- doubt-driven-development — Decisão de risco alto ou código que você não domina — revisão adversarial em contexto limpo.

Verificar

- `test-driven-development / tdd` — Escrever teste que falha primeiro, depois fazer passar (ver nota de duplicidade na seção 6).
- `browser-testing-with-devtools` — Verificação em runtime via Chrome DevTools MCP, se a fatia é browser-based.
- `debugging-and-error-recovery` — Algo quebrou — reproduzir → localizar → corrigir → proteger com teste.

Revisar

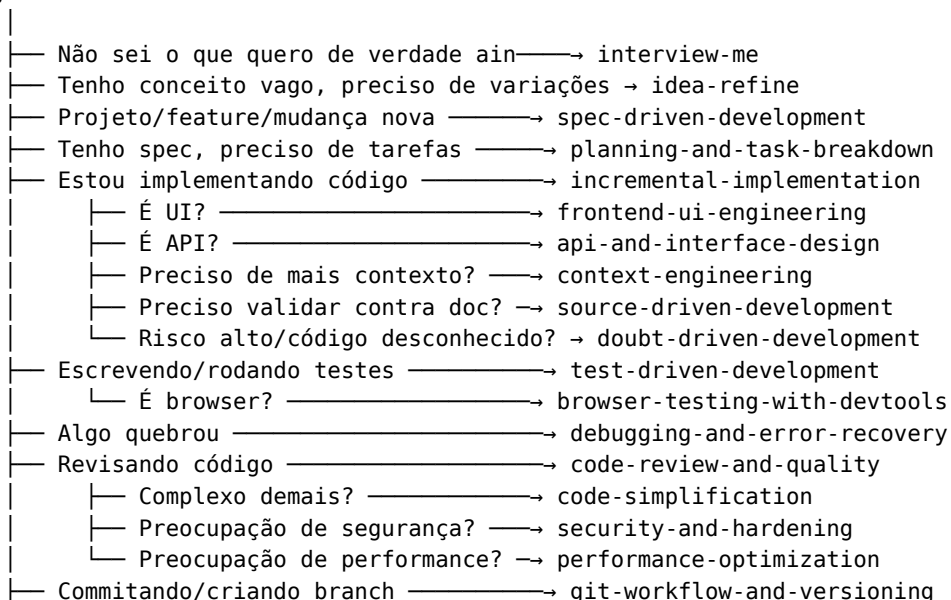
- `code-review-and-quality` — Revisão em 5 eixos antes de mergear (ver nota de duplicidade na seção 6).
- `code-simplification` — Reduzir complexidade desnecessária preservando comportamento.
- `security-and-hardening` — Preocupação de segurança — OWASP, validação de entrada, menor privilégio.
- `performance-optimization` — Preocupação de performance — medir antes de otimizar.

Entregar

- `git-workflow-and-versioning` — Organizar commits atômicos, histórico limpo.
- `ci-cd-and-automation` — Configurar/ajustar pipeline de qualidade automatizado.
- `deprecation-and-migration` — Aposentar sistema antigo e migrar usuários com segurança.
- `documentation-and-adrs` — Documentar o *porquê* da decisão, não só o *o quê*.
- `observability-and-instrumentation` — Adicionar logs estruturados/métricas/ alertas — roda em paralelo à construção, não depois.
- `shipping-and-launch` — Checklist pré-lançamento, monitoramento, plano de rollback.

4. Árvore de decisão rápida (“por onde eu começo?”)

Chegou uma tarefa



— Trabalho de pipeline CI/CD	→ ci-cd-and-automation
— Depreciando/migrando	→ deprecation-and-migration
— Escrevendo docs/ADRs	→ documentation-and-adrs
— Adicionando logs/métricas/alertas	→ observability-and-instrumentation
— Fazendo deploy/lançamento	→ shipping-and-launch

5. Cadeia completa — projeto do zero (greenfield)

Para uma feature/projeto novo inteiro, a sequência típica é:

1. interview-me → Extrair o que o operador quer de verdade
2. idea-refine → Refinar ideias vagas em variações concretas
3. spec-driven-development → Definir o que será construído + critério de aceite
4. planning-and-task-breakdown → Quebrar em pedaços pequenos e verificáveis
5. context-engineering → Carregar o contexto certo
6. source-driven-development → Verificar contra documentação oficial
7. incremental-implementation → Construir fatia por fatia
8. observability-and-instrumentation → Instrumentar enquanto constrói (roda em paralelo a 7-9, não depois)
9. doubt-driven-development → Contra-examinar decisões não-óbvias durante a construção
10. test-driven-development → Provar que cada fatia funciona
11. code-review-and-quality → Revisar antes de mergear
12. code-simplification → Reduzir complexidade desnecessária preservando comportamento
13. git-workflow-and-versioning → Histórico de commit limpo
14. documentation-and-adrs → Documentar as decisões
15. deprecation-and-migration → Aposentar sistemas antigos e migrar usuários (se aplicável)
16. shipping-and-launch → Deploy seguro

Nem toda tarefa precisa de todos os 16 passos. Uma correção de bug simples usa só: debugging-and-error-recovery → test-driven-development → code-review-and-quality.

Exemplo real desta sessão (greenfield dentro de um repo já existente)

O redesign visual do painel seguiu exatamente esse padrão, em escala reduzida:

1. interview-me painel → wizard multipasso — 4 rodadas de pergunta+palpite até confiança alta, terminando na decisão “colapsar setup + hierarquia visual + jobs sempre visível, sem wizard”.
 2. Plano salvo em melhorias/plano-painel-redesign-visual.md (não docs/intent/ — ver seção 1).
 3. Próximo passo natural: planning-and-task-breakdown (opcional aqui, pela simplicidade do escopo) → incremental-implementation direto.
-

6. Cadeia para projeto já iniciado (brownfield — o seu caso mais comum)

Quando o projeto já existe (como a Fábrica de Materiais de Comunicação, ou o painel dentro dela), você normalmente **entra no meio da cadeia**, não no início. Regra prática:

Sua situação agora	Onde entrar na cadeia
Vou pedir uma mudança e não tenho certeza do que quero	interview-me (sempre vale, mesmo em projeto maduro)
Já sei o que quero, é uma mudança pequena e óbvia	Pule direto para incremental-implementation
Já sei o que quero, é uma mudança grande/arriscada	spec-driven-development → planning-and-task-breakdown → incremental-implementation
Só quero adicionar um teste/corrigir um bug	debugging-and-error-recovery → test-driven-development → code-review-and-quality
Vou mexer numa API/contrato já usado por outra parte do sistema	api-and-interface-design antes de tocar no código
Vou revisar um branch/PR já pronto	code-review-and-quality (ou o par graph-powered, ver abaixo)
Vou desativar algo antigo (comando, endpoint, skill)	deprecation-and-migration

Sobreposições conhecidas (leia antes de escolher)

Este repo tem **dois conjuntos de skills que se sobrepõem** — o antigo (específico deste repo, muitos usando o MCP code-review-graph já construído) e o novo (genérico, do plugin Agent Skills). Use a tabela abaixo pra não invocar a errada:

Necessidade	Skill graph-powered (prefira dentro deste repo , se o grafo estiver atualizado)	Skill genérica (use se o grafo não existir/estiver desatualizado, ou fora deste repo)
Debugar	debug-issue, diagnosing-bugs, systematic-debugging	debugging-and-error-recovery
Revisar código	code-review, review-changes	code-review-and-quality
Testar	tdd, test-driven-development	(mesma dupla — ver nota abaixo)
Refatorar	refactor-safely	code-simplification
Navegar código	explore-codebase	context-engineering

Nota sobre TDD: tdd e test-driven-development coexistem no catálogo e cobrem o mesmo processo (red-green-refactor). Não há diferença prática relevante entre os dois neste repo hoje — escolha qualquer um, mas não rode os dois em sequência achando que são fases diferentes.

Regra prática: se code-review-graph já tem o grafo deste repo construído e atualizado (ver mcp__code-review-graph__list_graph_stats_tool), prefira as versões graph-powered — elas são

mais baratas em tokens e já conhecem a estrutura real do código. Se o grafo estiver desatualizado (o hook de SessionStart avisa isso — ver aviso “Graph was built on X mas você está em Y”) ou você estiver fora deste repo, use as versões genéricas.

7. Comportamentos centrais (valem em toda skill, sempre)

Estas regras vêm do próprio meta-skill using-agent-skills e não são opcionais:

1. **Exponha suposições antes de implementar algo não-trivial** — liste o que está assumindo sobre requisito/arquitetura/escopo antes de codar, não depois.
 2. **Gerencie confusão ativamente** — se encontrar requisitos conflitantes, pare e pergunte; não escolha uma interpretação e espere que esteja certa.
 3. **Discorde quando fizer sentido** — não é modo “sim-máquina”; aponte o problema, quantifique o custo concreto, proponha alternativa.
 4. **Simplicidade é obrigatória** — se construiu 1000 linhas e 100 bastavam, é falha, não sofisticação.
 5. **Disciplina de escopo** — toque só no que foi pedido; não “aproveite” pra limpar código adjacente sem aprovação explícita.
 6. **Verifique, não assuma** — toda skill tem um passo de verificação; “parece certo” nunca é suficiente, precisa de evidência (teste passando, build, dado de runtime).
-

8. Referência rápida (todas as skills, um-liner)

- interview-me — Extrai o que o operador quer de verdade antes de plano/spec/código existirem
- idea-refine — Refina ideias com pensamento divergente e depois convergente
- spec-driven-development — Requisitos e critério de aceite antes do código
- planning-and-task-breakdown — Decompõe em tarefas pequenas e verificáveis
- incremental-implementation — Fatias verticais finas, testando cada uma antes de expandir
- source-driven-development — Verifica contra documentação oficial antes de implementar
- doubt-driven-development — Revisão adversarial em contexto limpo de toda decisão não-trivial
- context-engineering — Contexto certo na hora certa
- frontend-ui-engineering — UI de qualidade de produção, com acessibilidade
- api-and-interface-design — Interfaces estáveis com contratos claros
- test-driven-development / tdd — Teste que falha primeiro, depois faz passar
- browser-testing-with-devtools — Verificação em runtime via Chrome DevTools MCP
- debugging-and-error-recovery — Reproduzir → localizar → corrigir → proteger
- code-review-and-quality — Revisão em 5 eixos com quality gates
- code-simplification — Preserva comportamento reduzindo complexidade desnecessária
- security-and-hardening — Prevenção OWASP, validação de entrada, menor privilégio
- performance-optimization — Mede primeiro, otimiza só o que importa
- git-workflow-and-versioning — Commits atômicos, histórico limpo
- ci-cd-and-automation — Quality gates automatizados a cada mudança
- deprecation-and-migration — Remove sistemas antigos e migra usuários com segurança
- documentation-and-adrs — Documenta o porquê, não só o o quê
- observability-and-instrumentation — Logs estruturados, métricas RED, traces, alertas por sintoma

- shipping-and-launch — Checklist pré-lançamento, monitoramento, plano de rollback
-

9. Notas específicas deste projeto (proj_fabrica-comunicacao)

- **Idioma:** todas as respostas em texto para o operador são em PT-BR, sempre — código/commits podem manter nomes técnicos em português já usados no repo.
- **Planos/intenções confirmados** (saída de interview-me ou de decisão de escopo fechada): sempre em melhorias/plano-<slug>.md, nunca em docs/intent/.
- **Manuais de uso** (como este arquivo): sempre em manuais/MANUAL_<NOME>.md.
- **Este manual não substitui** AGENTS.md/SPEC_COMANDOS.md — aqueles regem a Fábrica de Materiais de Comunicação em si (REGRA 3, REGRA 6, REGRA 9, etc.); este manual é sobre o framework de engenharia genérico usado para construir/manter o código do repo (painel, scripts, testes).